

Scaling

Vertical Scaling :- Increasing the power of single machine

You upgrade :- • More CPU, more RAM, faster SSD, better hardware.

• Instead of changing architecture you make the server stronger.

Ex:- Your DB runs on 8GB RAM & 4 CPU cores.

You upgrade to 64GB RAM & 16 CPU cores.

Note:- No architectural change required.

Limitations :-

- Hardware has limits
- Very expensive at high levels.
- Single point of failure
- Downtime during upgrades.
- You cannot scale infinitely upward.

Horizontal Scaling :- Adding more machines instead of upgrading one. Instead of one powerful server, we use multiple smaller servers. Instead of 1 server handling 10000 req. per second, use 10 servers handling 1000 each.

→ It provides :-

- Better scalability
- High availability
- Fault tolerance
- Cost flexibility.

How it works

Usually requires :-

- Load balancer
- Distributed cache
- DB replication
- Stateless application server

Flow → User → Load Balancer → Multiple App servers.

* DB Scaling :- Databases can also scale :-

1) Vertical :- Bigger DB machine

2) Horizontal :- Sharding & Replication.

Real system example

→ Startup Phase

- use vertical scaling
- Single DB
- Simple architecture

→ Growth Phase

- Add Load Balancer
- Add more app servers
- Add Redis
- Add read replicas.

→ Enterprise Phase

- Sharding
- Multi-region deployment.

Trade offs

Vertical Scaling

Pros:

- Simple
- Less complex

Cons:

- Limited growth
- Expensive
- Single point of failure

Horizontal Scaling

Pros:-

- Highly scalable
- Fault tolerant
- Flexible

Cons:-

- More complexity
- Distributed system challenges.
- Requires load balancing.

Capacity Estimation

→ Process of calculating expected system load in terms of traffic, storage, bandwidth and compute requirements to ensure the system can handle peak usage without failure.

It helps you decide :-

- * No. of servers
- * DB size
- * Cache size
- * Network requirements.

What do we estimate?

1) Traffic (RPS - Request per second)

Ex:- 1 Million daily users → each make 20 req/day
Total daily requests = 20 million.

Convert to RPS → $20M / (24 \times 60 \times 60) \approx 231$ RPS avg.

But peak traffic may be 5x or 10x. Plan for peak.

2) Storage:- Each user stores 1MB, 1 Million → 1TB storage
Add:- Logs, Backups, Indexes, Replication overhead.
Always add buffer (30-50%).

3) Bandwidth:- If response size = 100KB & RPS = 500
Bandwidth per second = $500 \times 100KB = 50MB/s$.
This impacts network & CDN design.

4) Compute:- Estimate CPU required based on :-
• Request complexity, DB queries, AI model calls, Background jobs

Why it is important?

• Helps you:-

- Choose vertical vs horizontal scaling
- Decide sharding strategy
- Design cache size
- Plan infrastructure budget.

Load Balancing

→ The process of distributing incoming requests across multiple servers to ensure no single server is overloaded.

Working :- User → Load Balancer → Server A/B/C

Load Balancing Algorithms

1) Round Robin :- Distribute requests sequentially.
Req 1 → Server A, Req 2 → Server B, Req 3 → Server C

2) Least Connections :- send request to server with fewest active connections.

• Better for uneven traffic.

3) Weighted Round Robin :- servers with higher capacity get more traffic.

• Usually when servers differ in power.

4) IP hash :- same client always routed to same server. useful for session based system.

Types :- 1) Layer 4 (Transport layer) → Based on IP + Port.
Faster, simpler.

2) Layer 7 (Application Layer) → Based on URL, Headers, Cookies.
Smarter, used in modern systems.

Example

→ E-commerce system :- User → Load Balancer
→ 10 App servers
→ Redis
→ Database

without load balancer :-

Horizontal scaling is impossible.

Consistent Hashing

* A technique for distributing data across multiple servers such that adding or removing a server affects only a small portion of keys.

Scenario:- Imagine having 4 servers (data distributed)

And you use simple hashing :- $\text{hash}(\text{Key}) \% 4$.

Now we add 1 new server. New formula = $\text{hash}(\text{Key}) \% 5$

Problem:- Almost all keys get reassigned which causes:-

→ Massive cache miss

→ Huge data reshuffling

→ Consistent hashing minimizes redistribution.

Working

Instead of servers in a list, we imagine a circular ring (hash ring).

Step 1:- Hash servers onto Ring. Each server gets a position on the ring based on $\text{hash}(\text{server_id})$.

Step 2:- Hash Keys onto Ring:- Each key also gets a position on the ring.

Step 3:- Assignment rule:- A Key is stored on the first server clockwise on the ring.

Example:- Suppose 3 servers A, B, C placed on ring. Key K1 is placed on ring. It moves clockwise until it finds first server.

The server stores it.

* Virtual Nodes

Prb:- Servers may not be evenly distributed on ring.

Sol:- Use virtual nodes. Place the server multiple times on the ring. This improves load balancing & even distribution.

Used in :- * Redis cluster * Cassandra * DynamoDB.

* Consistent Hashing Use Case

→ Distributed cache
→ CDN Routing

→ DB Sharding.

→ Microservice Load Balancing.

Trade Offs

Pros:-

- Minimal Data Movement
- Smooth Scaling
- Fault tolerance

Cons:-

- More complex than simple hashing
- Requires careful implementation.
- still needs replication for reliability.

Example

In Distributed caching system:

- 10 Redis nodes

If one node fails:-

Only Key assigned to that node are distributed,
Others remain unaffected and stable.

Scaling LLM Workloads

* Designing infrastructure to handle increasing AI inference requests efficiently without performance degradation.

It involves scaling:-

- Compute (CPU/GPU)
- Memory
- Throughput
- Model serving & Request handling.

Main challenges

- GPU is expensive
- Requests vary in size
- Token usage is unpredictable
- High latency and slow inference.

Techniques to scale LLM Workload

1) Horizontal Scaling

Instead of 1 model server, use multiple model servers behind Load balancer.

2) Request Batching :- Instead of processing 10 separate queries, Process them together on GPU. It improves GPU utilization.

3) Model Sharding :- For very large models, split model across multiple GPUs. It is used in:-

- Very large LLM deployments
- Enterprise AI infrastructure.

4) Caching (Important) :- Reduce LLM calls using:-

- Response, Embedding & retrieval caching.

5) Use smaller model when possible, large for complex tasks. This is model routing.

6) Async Processing :- For non-real time tasks:-

Use background jobs. Example:-

→ Document summarization for 500 PDFs

Cost Based Scaling

→ Scaling AI infrastructure in a way that optimizes performance while controlling operational costs.

Techniques for Cost-Based scaling

1) Intellegent Model Routing

Simple Query - cheap model , Complex Query - expensive

2) Aggressive Caching

Reuse respons.

3) Token optimisation :- Reduce prompt size , context length , unnecessary history.

4) Retrieval Optimization :- Better RAG retrieval → fewer documents → More relevant context.

5) Auto scaling GPU instances :- Scale GPU servers.

→ Up during peak → Down during low usage.

• Cloud based scaling.

6) Rate limiting & Quotas :- Limit free tier, users, Abuse, Overuse.

Example

• SaaS AI product

- Free users → smaller model
- Paid users → Larger model
- Cached FAQs served without LLM calls.
- Auto-scaling during business hours.